

have adjusted the choice of N to ensure that the optimal 2bit/param models are within a factor of 20 of each other in terms of model sizes.

Our results are presented in Figure 2 (in the main body, limited to models with accuracy $\leq 50\%$ for clarity) and in Figure 12 (including all models).

Furthermore, from the bit complexity lower bound (see Definition 4.1)

$$\underbrace{N \log_2 \frac{N_0}{e^{p_1}}}_{\text{name}} + \underbrace{NK \log_2 \frac{D^C}{e^{p_2}}}_{\text{value}} + \underbrace{KD \log_2 \frac{T^L}{De^{p_3}}}_{\text{diversity}} \quad (\text{A.1})$$

we also dissect how the three components contribute to this overall lower bound. As shown in Figure 13, although the “value” component typically dominates, for certain hyperparameter settings, the “name” or “diversity” components can also be significant. This underscores the importance of proving our Theorem 3.2 lower bound, which is a sum of all three terms.

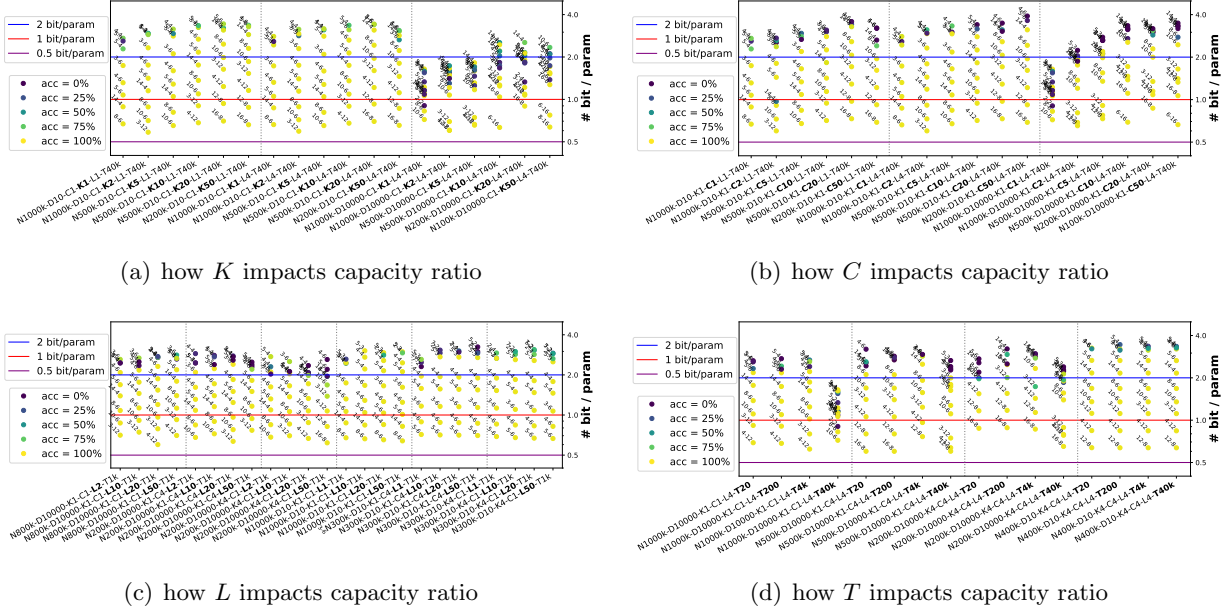
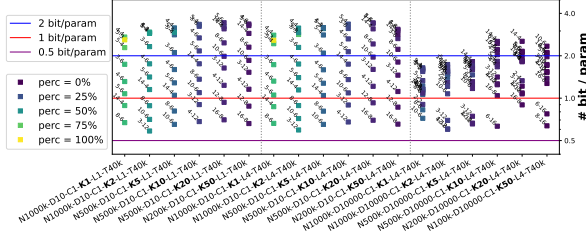
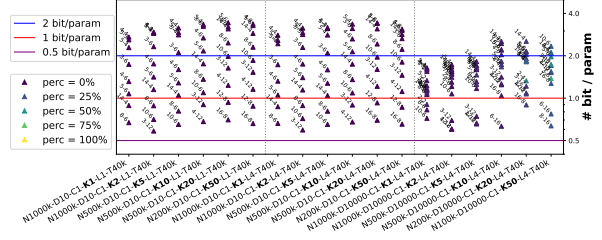


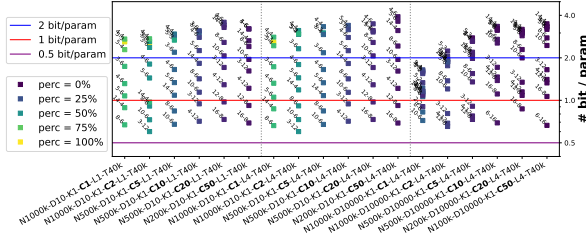
Figure 12: Same as Figure 2, but including models with accuracies below 50% (which may overlap with higher-accuracy models). The *peak* capacity ratios consistently exceed $R(F) \geq 2$.



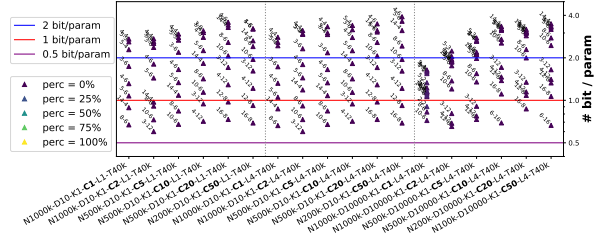
(a) percentage of bits belonging to “name” in (A.1)



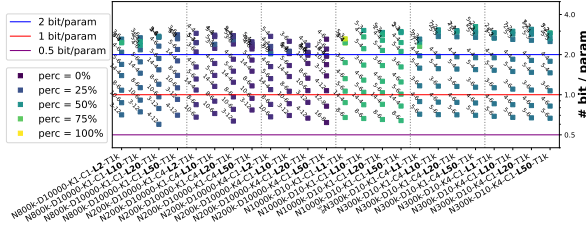
(b) percentage of bits belonging to “diversity” in (A.1)



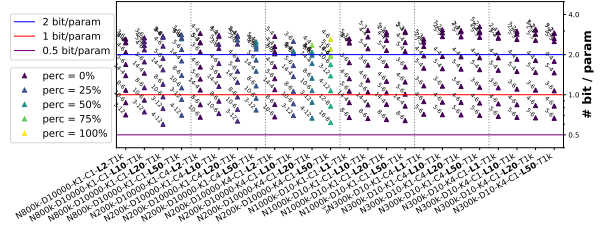
(c) percentage of bits belonging to “name” in (A.1)



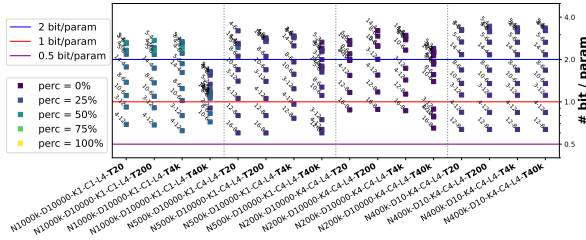
(d) percentage of bits belonging to “diversity” in (A.1)



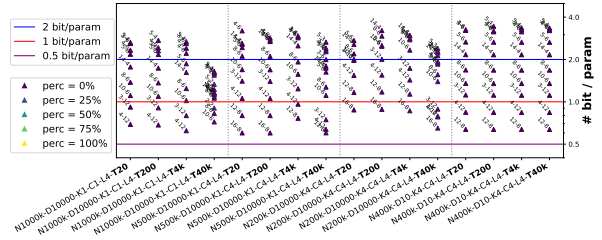
(e) percentage of bits belonging to “name” in (A.1)



(f) percentage of bits belonging to “diversity” in (A.1)



(g) percentage of bits belonging to “name” in (A.1)



(h) percentage of bits belonging to “diversity” in (A.1)

Figure 13: Breakdown of knowledge components in the parameterized bioD scaling law experiments, as shown in Figure 2. Refer to Equation (A.1) and the accompanying text.

B More on Model Architectures

We explore alternative architectural choices for language models.

LLaMA/Mistral. Notably, as of the writing of this paper, LLaMA [32, 33] and Mistral [19] stand out as popular, publicly-available large language models. We highlight their key architecture differences from GPT2 — which we define as having rotary embedding and no dropout.

1. LLaMA and Mistral employ MLP layers with gated activation, using $V(\sigma(W_1x) \cdot (W_2x))$ instead of $V\sigma(Wx)$. Shazeer [29] noted that gated activation appears to yield slightly better performance.
2. Unlike GPT2, which ties the weights of the embedding layer and the output (LMHead) layer, LLaMA and Mistral do not.
3. For a hidden dimension d , GPT2/LLaMA have $4d^2$ parameters in the attention layer and $8d^2$ in the MLP layer, whereas Mistral allocates a larger $10.5d^2$ for its MLP layer.
4. Mistral promotes group-query attention (e.g., using 4 groups, thus reducing the K/V matrices to $d^2/4$ in size), unlike GPT2. LLaMA does not favor multi-query attention unless in its very large models, such as the 70B variant.
5. LLaMA and Mistral utilize different tokenizers compared to GPT2, with Mistral’s tokenizer being nearly identical to LLaMA’s.
6. GPT2 employs $\sigma = \text{gelu}$, while LLaMA/Mistral use $\sigma = \text{silu}$.
7. GPT2 incorporates layer normalization with trainable bias, which LLaMA/Mistral do not.

Given these distinctions, for LLaMA models, we use the notation LLaMA- ℓ - h for ℓ layers, h heads, and $64h$ hidden dimensions; we omit group-query attention as LLaMA recommends it only for its 70B model. For Mistral, denoted as Mistral- ℓ - h , we enable group-query attention with 4 groups if $h = 0 \pmod{4}$, 1 group for odd h , or 2 groups otherwise.

GPT2 with Smaller MLP. Mistral has a larger MLP layer, and it is often believed that the MLP layer serves primarily for storing knowledge, in contrast to the Attention layer. But is this truly the case?

To delve into this, we examine GPT2_{1/4}, which is GPT2 with its MLP layer reduced from $d \rightarrow 4d \rightarrow d$ to $d \rightarrow d \rightarrow d$ (thus, 1/4 of its original size), and GPT2₀, which is GPT2 but without any MLP layer.

Experimental setups. Throughout this section, when presenting positive result (such as for GPT2) we try to stick to one fixed set of learning rate choices; but when presenting a negative result (such as for the LLaMA architecture), we present the best among three learning rate choices.

B.1 1000-Exposure Setting

In the 1000-exposure setting, we observe that the model architecture choices have a *negligible impact* on the scaling laws. The results for LLaMA, Mistral, GPT2₀, and GPT2_{1/4} architectures are presented in Figure 3, with their parameter choices discussed below.

Parameter 6 (Figure 3). In the 1000-exposure setting, for LLaMA/Mistral models we use similar parameters as specified in Parameter 1, but we select the best of three learning rates to better demonstrate that GPT2 performs *no worse* than *even the best tuned* LLaMA/Mistral models:

- For $N = 10K$, we use $wd = 0.02$, $lr = 0.0005/0.001/0.002$, and batch size 24 with fp16;
- For $N = 20K$, we use $wd = 0.02$, $lr = 0.0005/0.001/0.002$, and batch size 48 with fp16;

- For $N = 50K$, we use $wd = 0.02$, $lr = 0.0005/0.001/0.002$, and batch size 96 with fp16;
- For $N = 100K, 200K$, we use $wd = 0.02$, $lr = 0.0005/0.001/0.002$, and batch size 192 with fp16;
- For $N = 500K, 1M$, we use $wd = 0.01$, $lr = 0.0002/0.0003/0.0005$, and batch size 192 with fp16;
- For $N = 2M$, we use $wd = 0.005$, $lr = 0.0003/0.0005/0.001$, and batch size 1536 with bf16;
- For $N = 5M$, we use $wd = 0.002$, $lr = 0.0003/0.0005/0.001$, and batch size 1536 with bf16;
- For $N = 10M$, we use $wd = 0.001$, $lr = 0.0003/0.0005/0.001$, and batch size 1536 with bf16.

For GPT2₀ and GPT2_{1/4}, we use the same learning rates as specified in Parameter 1.

Remark B.1 (bf16 on gated MLP). As discussed in Section B.2, the training of LLaMA and Mistral architectures is less stable due to the use of GatedMLP, leading to the necessity of switching to (mixed-precision) bf16 training when required.

From Figure 3, it is evident that, except for tiny models, LLaMA, Mistral, GPT2₀, and GPT2_{1/4} architectures closely follow GPT2’s scaling law over 1000 exposures. For tiny models with $\leq 10M$ parameters, tying model weights increases their capacity (refer to Figure 3(c)). This indicates that the *2bit/param capacity ratio is a relatively universal law* among most typical (decoder-only) language model architectures.

B.2 100-Exposure Setting

The 100-exposure setting reveals more intriguing comparisons. We contrast GPT2 with various model architectures in Figure 4 and offer a detailed comparison between LLaMA and GPT2 architectures in Figure 5.

Figure 4(b) shows that the LLaMA architecture may lag behind GPT2’s scaling law by a factor of 1.3x, even for larger models.

We delve into the reasons behind this. By adjusting LLaMA’s architecture (e.g., switching GatedMLP back to normal MLP), as shown in Figure 5, we find that replacing LLaMA’s GatedMLP with a standard MLP is necessary to match GPT2’s scaling law. Notably, for a strong comparison, when using GatedMLP we select the best result from three learning rates, whereas for a standard MLP, akin to GPT2, we use a single learning rate. For smaller models, matching GPT2 requires tying model weights and adopting GPT2’s tokenizer, though this is less significant.²⁶

For other model architectures, Mistral, GPT2₀, and GPT2_{1/4}, their scaling laws in the 100-exposure setting are presented in Figure 4. Figure 4(c) confirms that the Mistral architecture also underperforms GPT2 due to its use of gated MLP. Figure 4(d) reveals that reducing GPT2_{1/4}’s MLP layer size by a quarter has a *negligible impact* on model capacity. However, removing the MLP layers entirely in GPT2₀ significantly reduces the model’s capacity, see Figure 4(e).

The 100-exposure setting represents an “insufficient training” paradigm. Thus, the comparisons are not about one architecture being strictly worse than another (as they achieve similar capacity ratios in a 1000-exposure setting, as shown in Figure 3). Our findings indicate that some architectures are *noticeably easier to train (thus learn knowledge faster)*:

- The GatedMLP architecture *slows down* the model’s learning speed, and we observe less stable training with its use.²⁷
- Removing MLP layers entirely *slows down* the model’s learning speed, whereas adjusting the size of MLP layers (e.g., from $8d^2$ to $10.5d^2$ or down to $2d^2$) may not have a significant impact.

²⁶The influence of the tokenizer on model capacity is noteworthy. For instance, LLaMA/Mistral tokenizers tend to split birthday years into single-digit tokens, slightly slowing the training of smaller models, whereas the GPT2Tokenizer uses a single token for the birth years such as 1991.

²⁷For example, mixed-precision fp16 training can sometimes fail for LLaMA/Mistral models smaller than 100M; hence, we use mixed-precision bf16 instead. Conversely, GPT2 models up to 1B can be trained with fp16.